

Universidade Católica de Brasília

Engenharia de Software

Documento de Arquitetura do Sistema

UniCarona

Plataforma de Caronas Solidárias para a Comunidade Acadêmica

GRUPO 05

Versão 1.0

Brasília, 2026

Revisões do Documento

Toda modificação no documento deve constar nesta tabela para garantir o controle de versão e rastreabilidade das alterações realizadas.

Versão	Data	Autores	Observações
1.0	Abril/2026	Grupo 05	Versão inicial do documento de arquitetura

1. Introdução

1.1 Finalidade

Este documento tem como finalidade descrever a arquitetura do sistema UniCarona de forma detalhada e estruturada, servindo como referência técnica para a equipe de desenvolvimento, stakeholders e demais envolvidos no projeto. Apresenta as decisões arquiteturais tomadas, os estilos e padrões adotados, os componentes e suas responsabilidades, as tecnologias previstas e as justificativas para as escolhas realizadas.

O documento evolui e complementa as definições introduzidas no Documento de Visão, aprofundando os aspectos de implementação e estrutura do sistema para guiar as fases de desenvolvimento, testes e manutenção.

1.2 Escopo

Este documento cobre a arquitetura completa da plataforma UniCarona, um aplicativo móvel multiplataforma (Android e iOS) voltado para a intermediação de caronas solidárias exclusivamente para a comunidade acadêmica da Universidade Católica de Brasília. A arquitetura abrange:

- A camada de apresentação (aplicativo móvel desenvolvido em Flutter/Dart).
- A camada de negócio (servidor backend desenvolvido com NestJS e TypeScript).
- A camada de dados (banco de dados PostgreSQL com suporte geoespacial).
- As integrações com serviços externos (APIs de mapas, reconhecimento facial e notificações).
- Os padrões de segurança, comunicação e qualidade adotados.

1.3 Definições, Acrônimos e Abreviações

- API (Application Programming Interface): Interface de comunicação entre sistemas de software.
- REST / RESTful: Estilo arquitetural para APIs baseado em recursos e operações HTTP padronizadas.
- JWT (JSON Web Token): Padrão para transmissão segura de informações de autenticação entre partes.
- ORM (Object-Relational Mapping): Técnica de mapeamento entre objetos de código e tabelas de banco de dados.
- GPS (Global Positioning System): Sistema de posicionamento global via satélite.
- MVP (Minimum Viable Product): Versão mínima viável do produto com funcionalidades essenciais.
- SDK (Software Development Kit): Conjunto de ferramentas para desenvolvimento de software.
- TLS/SSL: Protocolos de criptografia para comunicação segura na internet.
- Matching: Algoritmo que cruza dados de rota e horário para conectar motoristas e passageiros compatíveis.
- Facial ID: Tecnologia de reconhecimento facial biométrico utilizada para validação de identidade.

2. Descrição Geral da Arquitetura

2.1 Visão Geral

O UniCarona é um sistema distribuído baseado na arquitetura Cliente-Servidor em três camadas (Three-Tier Architecture), onde o aplicativo móvel (cliente) se comunica com o servidor de aplicação (backend) por meio de uma API RESTful, e o servidor persiste os dados em um banco de dados relacional dedicado.

A plataforma foi concebida para ser um facilitador digital de logística social dentro do ambiente acadêmico. Diferentemente de aplicativos comerciais de transporte, o UniCarona é um sistema independente, sem integração com sistemas de transporte público e sem módulos de pagamento eletrônico, operando estritamente como uma rede de caronas solidárias.

A arquitetura prioriza três pilares fundamentais: segurança institucional (validação rigorosa de identidade e rastreabilidade completa das viagens), usabilidade (interface intuitiva e multiplataforma) e escalabilidade (estrutura modular preparada para crescimento gradual da base de usuários).

2.2 Contexto do Sistema

O sistema opera em um contexto onde estudantes universitários da UCB precisam se deslocar diariamente entre suas residências nas cidades satélites do Distrito Federal e o campus, enfrentando problemas de transporte público limitado, custos elevados e desgaste físico. O UniCarona atua como uma camada de apoio à mobilidade acadêmica, conectando alunos motoristas com vagas ociosas a passageiros com trajetos compatíveis.

O diagrama de contexto a seguir ilustra as fronteiras do sistema e suas interações com atores externos:

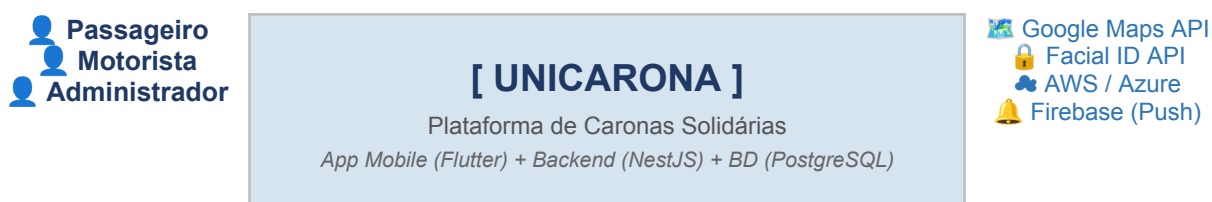


Figura 1 – Diagrama de Contexto do Sistema UniCarona

3. Estilo Arquitetural Adotado

3.1 Arquitetura Cliente-Servidor em Três Camadas

O UniCarona adota o estilo arquitetural Cliente-Servidor com separação em três camadas lógicas distintas (Three-Tier Architecture). Esse modelo é amplamente consolidado no desenvolvimento de aplicativos móveis e oferece clara separação de responsabilidades, facilitando manutenção, testes e escalabilidade independente de cada camada.

Camada 1 — Apresentação	Camada 2 — Negócio	Camada 3 — Dados
Aplicativo Móvel Flutter / Dart <i>Interface do usuário, interações e visualizações</i>	Servidor de Aplicação NestJS / TypeScript <i>Regras de negócio, algoritmos, segurança e APIs</i>	Banco de Dados PostgreSQL <i>Persistência de dados, rotas, usuários e histórico</i>

Figura 2 – Visão em Camadas da Arquitetura UniCarona

3.2 Padrão de Comunicação: API RESTful

A comunicação entre o aplicativo móvel (cliente) e o servidor (backend) segue o padrão de arquitetura API RESTful. As requisições são realizadas via protocolo HTTPS, trafegando dados estruturados no formato JSON. Esse padrão garante interoperabilidade, facilidade de teste e separação clara entre frontend e backend, permitindo que ambos evoluam de forma independente.

3.3 Padrão de Organização do Backend: Módulos Orientados a Domínio

O servidor backend é organizado seguindo o padrão arquitetural de módulos orientados a domínio, conforme proposto pelo framework NestJS. Cada funcionalidade principal do sistema (autenticação, rotas, matching, viagens, avaliações, chat) é encapsulada em um módulo independente com seus próprios controllers, services e repositories, promovendo alta coesão e baixo acoplamento entre as partes do sistema.

4. Principais Componentes e Responsabilidades

4.1 Visão Geral dos Componentes

O sistema UniCarona é composto por dez componentes principais, distribuídos entre as camadas de apresentação, negócio e dados. A tabela a seguir apresenta cada componente, sua camada de atuação e sua responsabilidade no sistema:

Componente	Camada	Responsabilidade
Módulo de Cadastro e Autenticação	Backend / Frontend	Gerencia o registro de usuários, validação de CPF, e-mail institucional, reconhecimento facial (Facial ID) e geração de tokens JWT para sessões.
Módulo de Gestão de Rotas	Backend / Frontend	Permite que motoristas cadastrem rotas e horários frequentes. Passageiros visualizam rotas disponíveis com filtros por região, horário e destino.
Algoritmo de Matching	Backend	Núcleo lógico do sistema. Cruza os dados de rota e horário do motorista com a solicitação do passageiro usando dados geoespaciais do PostgreSQL para sugerir caronas compatíveis.
Módulo de Rastreamento em Tempo Real	Backend / Frontend	Monitora a posição GPS da viagem em andamento e exibe no mapa interativo para passageiro, motorista e administradores.
Módulo de Histórico de Viagens	Backend	Registra e persiste todas as viagens realizadas para fins de rastreabilidade e segurança institucional.
Módulo de Avaliação Bidirecional	Backend / Frontend	Permite que motorista e passageiro se avaliem mutuamente com notas e comentários ao término de cada viagem.
Chat Interno	Backend / Frontend	Comunicação privada entre motorista e passageiro para combinação de pontos de encontro, sem exposição de dados pessoais como telefone.
Sistema de Notificações Push	Backend / Mobile	Envia alertas para os usuários sobre confirmações de carona, horários de saída e outras atualizações relevantes.

Componente	Camada	Responsabilidade
Painel Administrativo	Backend / Frontend Web	Ferramenta de gestão para administradores da plataforma monitorarem usuários, rotas, viagens e garantirem conformidade com as políticas de uso.
API Gateway / RESTful	Backend	Camada de comunicação entre o aplicativo móvel e o servidor. Padroniza todas as requisições no formato JSON sobre HTTPS.

Tabela 1 – Componentes do Sistema UniCarona e suas Responsabilidades

4.2 Detalhamento dos Componentes Críticos

4.2.1 Módulo de Cadastro e Autenticação

Este é o componente de maior criticidade do sistema, pois é a porta de entrada da plataforma e garante a exclusividade institucional do UniCarona. O processo de cadastro é composto pelas seguintes etapas sequenciais: (1) fornecimento de dados pessoais (nome, CPF, e-mail institucional e senha), (2) validação do CPF junto a serviços de verificação, (3) comprovação do vínculo universitário via e-mail institucional, (4) captura e validação biométrica pelo Facial ID, e (5) geração do token JWT para acesso autenticado à plataforma. Sem a conclusão dessas etapas, o usuário não pode acessar nenhuma outra funcionalidade do sistema.

4.2.2 Algoritmo de Matching

O algoritmo de matching é o núcleo funcional do UniCarona e o diferencial da plataforma em relação a soluções informais de carona. Seu funcionamento baseia-se no cruzamento de três variáveis principais: compatibilidade geoespacial (proximidade entre origem do passageiro e rota do motorista, utilizando o Geography data type do PostgreSQL), compatibilidade de horários (janela de tolerância configurável entre o horário de saída do motorista e a necessidade do passageiro) e disponibilidade de vagas (número de assentos disponíveis no veículo do motorista). O resultado é uma lista ranqueada de caronas sugeridas ao passageiro, ordenada por grau de compatibilidade.

4.2.3 Módulo de Rastreamento em Tempo Real

Para garantir a segurança institucional das viagens, o sistema implementa rastreamento em tempo real durante toda a duração da carona. O dispositivo do motorista transmite continuamente sua posição GPS ao servidor via WebSocket, que distribui essa informação ao passageiro e, em casos configurados, aos administradores da plataforma. Todo o trajeto é registrado e armazenado no histórico de viagens para fins de auditoria e segurança.

5. Diagrama de Arquitetura

5.1 Diagrama de Implementação

O diagrama a seguir apresenta a visão de implementação da arquitetura do UniCarona, detalhando os artefatos tecnológicos de cada camada e os protocolos de comunicação entre eles:

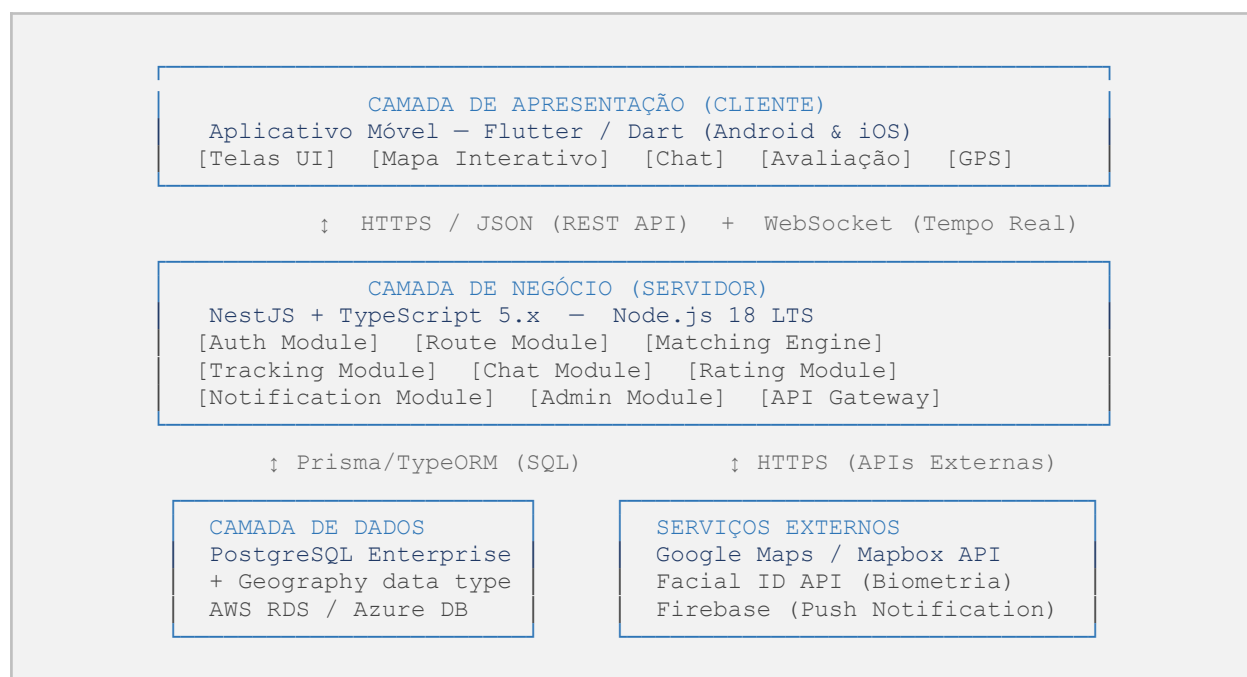


Figura 3 – Diagrama de Implementação da Arquitetura UniCarona

5.2 Fluxo Principal de Dados

O fluxo principal de dados do sistema para a funcionalidade central de matching de carona segue a seguinte sequência: (1) o passageiro informa sua origem, destino e horário no aplicativo mobile; (2) a requisição é enviada via HTTPS à API RESTful do servidor NestJS; (3) o módulo de matching consulta o banco PostgreSQL utilizando queries geoespaciais para encontrar motoristas com rotas e horários compatíveis; (4) os resultados são retornados ao aplicativo em formato JSON; (5) o passageiro seleciona uma carona e o motorista recebe uma notificação push via Firebase; (6) após confirmação mútua, o rastreamento em tempo real é iniciado via WebSocket.

6. Tecnologias Previstas

6.1 Stack Tecnológica Completa

A tabela a seguir apresenta todas as tecnologias previstas para o desenvolvimento e operação do UniCarona, organizadas por categoria, com a respectiva justificativa para cada escolha:

Categoria	Tecnologia	Justificativa
Frontend / Mobile	Flutter + Dart 3.x	Desenvolvimento multiplataforma (Android e iOS) com uma única base de código, garantindo menor custo de manutenção e UI consistente.
Backend	Node.js 18 LTS + NestJS + TypeScript 5.x	Stack madura com escalabilidade modular. TypeScript garante tipagem estática e redução de bugs. NestJS provê organização orientada a módulos e domínios.
Banco de Dados	PostgreSQL Enterprise	Banco relacional robusto com suporte nativo a dados geoespaciais (Geography data type), essencial para cálculo de rotas e matching por proximidade.
ORM	Prisma ou TypeORM	Abstração do banco de dados, facilitando migrações, relacionamentos e manutenção do esquema de dados.
Autenticação	JWT (JSON Web Token)	Padrão stateless para gestão segura de sessões, compatível com APIs RESTful e sem necessidade de estado no servidor.
Biometria / Identidade	Facial ID API (terceiros)	Validação biométrica exigida pelo escopo de segurança institucional. Integração via API externa especializada.
Mapas e Geolocalização	Google Maps API ou Mapbox	APIs consolidadas para exibição de rotas, geocodificação e cálculo de proximidade entre origem e destino.
Comunicação em Tempo Real	Socket.io ou Firebase	Necessário para o rastreamento em tempo real da carona e o chat interno entre motorista e passageiro.
Infraestrutura Cloud	AWS ou Azure	Hospedagem do backend e banco de dados com alta disponibilidade, escalabilidade e suporte a picos de acesso nos horários de aula.
Protocolo de Segurança	HTTPS + TLS/SSL	Obrigatório para tráfego de dados sensíveis como CPF e validação biométrica, conforme as restrições técnicas definidas.
IDE / Ambiente de Dev	VS Code ou Android Studio	Ferramentas com suporte oficial para Flutter, Dart, TypeScript e plugins de produtividade.

Tabela 2 – Stack Tecnológica Completa do UniCarona

6.2 Premissas de Ambiente de Desenvolvimento

6.2.1 Máquina do Desenvolvedor

- SDK Flutter: Versão estável (Stable Channel) para desenvolvimento da interface multiplataforma.
- Linguagem Dart: Versão 3.x ou superior.
- IDE: Visual Studio Code ou Android Studio com plugins oficiais do Flutter e Dart.
- Hardware: Mínimo de 8GB RAM (16GB recomendado) e processador com suporte a virtualização (VT-x ou AMD-V) para emulação de dispositivos Android e iOS.

6.2.2 Ambiente de Servidor

- Ambiente de Execução: Node.js versão 18.x (LTS) ou superior.
- Framework Backend: NestJS para garantir escalabilidade modular e organização por domínios.
- Linguagem: TypeScript 5.x para tipagem estática e redução de erros em tempo de execução.
- Gerenciador de Pacotes: NPM ou Yarn.

6.2.3 Banco de Dados

- Servidor: PostgreSQL Enterprise.
- ORM: Prisma ou TypeORM para a ponte entre o servidor Node.js e o PostgreSQL.
- Requisito Especial: Suporte obrigatório ao tipo de dado geoespacial (Geography data type) para o cálculo preciso das rotas e o algoritmo de matching por proximidade.

7. Justificativa das Decisões Arquiteturais

7.1 Escolha do Estilo Arquitetural: Cliente-Servidor em Três Camadas

A arquitetura Cliente-Servidor em três camadas foi escolhida por ser o modelo mais adequado para sistemas de aplicativos móveis que requerem processamento centralizado de dados sensíveis e algoritmos complexos no servidor. Essa separação garante que a lógica de segurança (autenticação, validação de identidade, matching) permaneça no servidor controlado pela equipe, impedindo que usuários mal-intencionados manipulem o processo pelo lado do cliente. Adicionalmente, a separação em camadas permite a escalabilidade independente de cada nível: caso a demanda aumente, é possível escalar apenas o servidor de aplicação sem modificar o aplicativo mobile ou o banco de dados.

7.2 Escolha do Flutter/Dart para o Frontend

O Flutter foi adotado para o desenvolvimento do aplicativo móvel porque permite a criação de uma única base de código compatível com Android e iOS simultaneamente. Dado que a comunidade acadêmica utiliza ambas as plataformas, o desenvolvimento nativo separado duplicaria o esforço e o custo da equipe. Adicionalmente, o Flutter oferece performance próxima ao nativo, suporte aos padrões Material Design e Cupertino, e um ecossistema maduro para integração com GPS, mapas e notificações push.

7.3 Escolha do NestJS e TypeScript para o Backend

O NestJS foi escolhido por sua arquitetura modular inspirada no Angular, que força uma organização clara do código em módulos, controllers, services e repositories. Essa estrutura facilita a manutenção e a evolução do sistema por diferentes membros da equipe ou por futuros mantenedores da universidade. O TypeScript garante tipagem estática rigorosa, reduzindo significativamente a ocorrência de bugs em tempo de execução e melhorando a legibilidade e documentação implícita do código.

7.4 Escolha do PostgreSQL com Dados Geoespaciais

O PostgreSQL Enterprise foi selecionado como banco de dados por ser um dos sistemas relacionais mais robustos e maduros disponíveis, com suporte nativo ao tipo de dado Geography e à extensão PostGIS. Essa capacidade geoespacial é absolutamente essencial para o algoritmo de matching do UniCarona, que precisa calcular proximidade entre coordenadas GPS, verificar se uma rota de motorista passa próximo à origem de um passageiro, e ordenar resultados por distância. Nenhum outro banco de dados oferece essa combinação de confiabilidade relacional e poder geoespacial de forma tão integrada.

7.5 Escolha da API RESTful como Protocolo de Comunicação

O padrão RESTful foi adotado para a comunicação entre o aplicativo móvel e o servidor por ser amplamente consolidado, bem documentado e de fácil testabilidade. APIs REST são stateless (sem estado no servidor), o que simplifica a escalabilidade horizontal e o

balanceamento de carga. O formato JSON escolhido para a troca de dados é leve, legível e amplamente suportado tanto pelo Flutter/Dart quanto pelo NestJS/TypeScript. Para as funcionalidades que exigem comunicação em tempo real (rastreamento de viagem e chat), o WebSocket complementa a REST API sem substituí-la.

7.6 Escolha do JWT para Autenticação

O JSON Web Token (JWT) foi adotado como mecanismo de autenticação por ser um padrão stateless que dispensa o armazenamento de sessões no servidor. Em um aplicativo móvel, onde o usuário pode alternar entre conexões de rede (Wi-Fi, 4G, 5G), tokens JWT garantem que a sessão permaneça válida independentemente da conexão, desde que o token não tenha expirado. Adicionalmente, os tokens são assinados criptograficamente, impedindo falsificação, e podem carregar informações do usuário (claims) sem necessidade de consultas adicionais ao banco de dados em cada requisição.

7.7 Adoção de Serviços Externos para Mapas e Biometria

A decisão de utilizar APIs externas para mapas (Google Maps ou Mapbox) e reconhecimento facial (Facial ID) baseia-se no princípio de não reinventar a roda para funcionalidades altamente especializadas. Desenvolver um sistema de mapas ou de biometria facial do zero demandaria recursos, tempo e expertise que extrapolam o escopo do projeto. Os serviços escolhidos são líderes de mercado, oferecem alta precisão, SLAs de disponibilidade garantidos e possuem SDKs oficiais compatíveis com as tecnologias adotadas no projeto. A restrição conhecida é a dependência de limites de requisição e disponibilidade desses provedores externos.

8. Manual de Arquitetura

8.1 Visão de Implementação

O sistema UniCarona é estruturado seguindo uma arquitetura Cliente-Servidor em camadas, onde o aplicativo móvel (cliente) se comunica com o servidor (backend) exclusivamente por meio de uma API RESTful, trafegando dados no formato JSON sob protocolo HTTPS. A implementação está dividida em três camadas principais:

- Camada de Apresentação (Frontend): Desenvolvida com Flutter/Dart, responsável por toda a interface gráfica do usuário, compatível com Android e iOS. É nessa camada que ocorrem as interações do usuário: cadastro, visualização de rotas, solicitação de carona, chat interno e mapa de rastreamento.
- Camada de Negócio (Backend): Desenvolvida com Node.js 18.x (LTS), framework NestJS e linguagem TypeScript 5.x. Responsável por toda a lógica de negócio, incluindo o algoritmo de matching, validação de identidade, geração e validação de tokens JWT e comunicação com APIs externas.
- Camada de Dados: Banco de dados PostgreSQL com suporte ao tipo de dado geoespacial (Geography data type), acessado via ORM (Prisma ou TypeORM). Responsável pela persistência de todos os dados: usuários, rotas, viagens, avaliações e histórico.

8.2 Mecanismos Arquiteturais

A tabela a seguir apresenta o mapeamento completo dos mecanismos arquiteturais do UniCarona, relacionando o mecanismo de análise (necessidade identificada), o mecanismo de design (solução conceitual) e o mecanismo de implementação (tecnologia adotada):

Mecanismo de Análise	Mecanismo de Design	Mecanismo de Implementação
Persistência de Dados	Banco de Dados Relacional com suporte geoespacial	PostgreSQL + Geography data type
Comunicação Cliente-Servidor	API RESTful stateless	NestJS + JSON over HTTPS
Segurança de Autenticação	Token de Sessão Stateless	JWT (JSON Web Token) + TLS/SSL
Validação de Identidade	Biometria Facial + Validação Documental	Facial ID API (serviço externo) + CPF
Geolocalização e Rotas	Dados Geoespaciais e Mapas Interativos	Google Maps/Mapbox API + PostGIS
Interface do Usuário	Design Multiplataforma Adaptativo	Flutter/Dart (Material Design + Cupertino)
Lógica de Negócio	Módulos Orientados a Domínio	NestJS + TypeScript 5.x
Algoritmo de Matching	Cruzamento de Rotas e Horários	Algoritmo customizado em TypeScript

Mecanismo de Análise	Mecanismo de Design	Mecanismo de Implementação
Comunicação em Tempo Real	WebSocket / Notificações Push	Socket.io / Firebase Cloud Messaging
ORM / Mapeamento Objeto-Relacional	Abstração de Acesso ao Banco	Prisma ORM ou TypeORM

Tabela 3 – Mecanismos Arquiteturais do Sistema UniCarona

9. Restrições e Padrões Técnicos

9.1 Padrões Aplicáveis

- **A interface deve adotar as diretrizes do Material Design (Android) e Cupertino (iOS), garantindo usabilidade familiar em ambas as plataformas.:** Padrões de Interface (UI/UX)
- **A integração entre app e servidor utiliza o padrão API RESTful, com dados trafegados em formato JSON.:** Arquitetura de Comunicação
- **O tráfego de dados sensíveis (CPF, biometria) deve ser feito obrigatoriamente via HTTPS (TLS/SSL). Sessões gerenciadas com JWT.:** Padrões de Segurança
- **TypeScript com tipagem estrita no backend. Convenções oficiais do Dart no frontend.:** Qualidade e Padrão de Código

9.2 Restrições Aplicáveis

- **O projeto está limitado às ferramentas homologadas: Flutter/Dart (app), NestJS/TypeScript (backend) e PostgreSQL com Geography data type (banco de dados).:** Stack Tecnológica Engessada
- **O funcionamento depende da disponibilidade dos serviços de mapas (Google Maps/Mapbox) e da API de reconhecimento facial.:** Dependência de APIs Externas
- **O aplicativo não opera em modo offline. É requisito irrevogável a conexão com internet (3G/4G/5G) e GPS ativo.:** Conectividade Obrigatória
- **O sistema não pode possuir integrações com gateways de pagamento eletrônico, pois caracteriza exclusivamente carona solidária.:** Restrição de Módulos Financeiros

10. Precedência e Prioridades dos Componentes

A tabela a seguir apresenta a ordem de implementação dos componentes arquiteturais, alinhada com as entregas planejadas e a prioridade de cada funcionalidade para o funcionamento do sistema:

No.	Componente Arquitetural	Prioridade	Entrega
01	Módulo de Cadastro e Autenticação (CPF + Facial ID + JWT)	Crítico	Entrega 1.0 (MVP)
02	Módulo de Gestão de Rotas (Motorista)	Crítico	Entrega 1.0 (MVP)
03	Solicitação de Carona (Passageiro)	Crítico	Entrega 1.0 (MVP)
04	Algoritmo de Matching (Geoespacial)	Crítico	Entrega 1.0 (MVP)
05	Módulo de Rastreamento em Tempo Real	Importante	Entrega 1.1 (Segurança)
06	Módulo de Histórico de Viagens	Importante	Entrega 1.1 (Segurança)
07	Sistema de Avaliação Bidirecional	Importante	Entrega 2.0 (Social)
08	Chat Interno	Útil	Entrega 2.0 (Social)
09	Painel de Estatísticas de Sustentabilidade	Útil	Entrega 3.0 (Engajamento)
10	Sistema de Notificações Push	Útil	Entrega 3.0 (Engajamento)

Tabela 4 – Precedência e Prioridades dos Componentes do UniCarona

10.1 Justificativa da Precedência

Os componentes marcados como Críticos para a Entrega 1.0 (MVP) foram priorizados por representarem o núcleo funcional sem o qual a plataforma não cumpre sua proposta de valor básica. Sem autenticação segura e sem o algoritmo de matching, o UniCarona não se diferencia de grupos de mensagens informais. Os componentes de rastreamento e histórico foram priorizados na Entrega 1.1 por garantirem a segurança institucional que justifica a exclusividade acadêmica da plataforma. Os demais componentes compõem camadas de experiência e engajamento que potencializam o uso, mas não são bloqueadores para o lançamento inicial.

Referências

LIMA, Patrick; LIMA, Vitor. RideUFF: Desenvolvimento de um aplicativo de carona solidária para a comunidade da Universidade Federal Fluminense. Niterói: UFF, 2019.

SOUZA, J. A. et al. Carona Solidária: Uma Alternativa de Mobilidade e suas Vantagens no Ambiente Acadêmico. Revista de Gestão e Tecnologia (IFES), 2018.

FERREIRA, L. M.; GOMES, R. S. Adoção de serviços de carona solidária: análise dos motivadores e barreiras entre usuários urbanos. Revista Gestão e Desenvolvimento (Feevale), v. 15, n. 2, 2018.

NestJS Documentation. Disponível em: <https://docs.nestjs.com>. Acesso em: Abril 2026.

Flutter Documentation. Disponível em: <https://flutter.dev/docs>. Acesso em: Abril 2026.

PostgreSQL Documentation — PostGIS Geography Type. Disponível em: <https://www.postgresql.org>. Acesso em: Abril 2026.